Here is the clean, polished English translation of your note, keeping all math and structure intact.

SGD Optimizer & Learning Rate Scheduling

1. Why Optimization Matters

Why are optimizers so critical?

Training a deep learning model is essentially a high-dimensional, non-convex optimization problem:
• Parameter dimensions range from tens of thousands to billions
• The loss landscape contains many saddle points and twisted valleys
• Gradients may change rapidly and inconsistently
• Training is highly sensitive to the learning rate and can easily diverge

The optimizer determines whether the model can actually learn.
SGD is the foundation of all modern optimizers.

2. Gradient Descent → SGD: From Deterministic to Stochastic

2.1 Empirical Risk Minimization (ERM)

Given a dataset:

$$\mathcal{D} = (x_i, y_i)_{i=1}^{N}$$

Empirical risk:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(x_i; \theta), y_i)$$

2.2 Full Gradient Descent

Every update:

$$\theta_{t+1} = \theta_t - \eta, \nabla_\theta L(\theta_t)$$

Drawbacks:
• Every update requires scanning the whole dataset (very slow)

• Sensitive to the geometry of the loss surface; easily gets stuck in sharp curved valleys
• Not feasible for large datasets (e.g., ImageNet)

→ Hence the need for SGD.

2.3 Stochastic Gradient Descent (SGD)

Use a mini-batch instead of the full dataset:

$$\theta_{t+1} = \theta_t - \eta, \nabla_\theta \ell(x_b, y_b; \theta_t)$$

Mini-batches typically contain 32, 64, 128 samples, or more.

Important properties of SGD:

1. High noise (high variance)
   Gradient direction becomes unstable, but this “noise” has a surprising effect:
   SGD escapes saddle points more easily than GD.
2. Fast updates
   No need to compute gradients over the entire dataset.
3. Good generalization
   SGD noise behaves like an implicit regularizer.

2.4 Why Does Noise Help? (Extremely Important Engineering Intuition)

Deep neural networks have many saddle points, not local minima.

Problems with GD:
• Near a saddle point, gradients become very small
• Updates stall and training stops improving

SGD noise “shakes” you off the saddle and allows continued descent.

A geometric intuition:
• Imagine standing at the peak of a saddle-shaped surface
• Full GD just stays stuck
• SGD’s randomness nudges you off the peak

3. Weight Decay

3.1 L2 Regularization

$$L_{\text{total}} = L(\theta) + \lambda|\theta|^2$$

Gradient:

$$\nabla L_{\text{total}} = \nabla L + \lambda\theta$$

3.2 SGD Update with (Coupled) Weight Decay

$$\theta_{t+1} = \theta_t - \eta(\nabla L + \lambda\theta_t)$$

Geometric meaning:

• Large parameters receive larger penalization

• Equivalent to continuously “pulling the parameters toward zero”

• Encourages smaller weights → smoother models → improved generalization

4. Gradient Explosion & Gradient Clipping

4.1 Causes of Exploding Gradients

• Recurrent connections (RNNs): repeatedly multiplying Jacobians, exponential blow-up

• Deep networks: chain rule multiplies many factors

• Learning rate too high

• Poor weight initialization

4.2 Clip-by-Value

$$g \leftarrow \text{clamp}(g, -c, c)$$

Simple but not commonly used.

4.3 Clip-by-Norm (Global Gradient Clipping)

Compute the total gradient norm:

$$|g| = \sqrt{\sum_i |g_i|^2}$$

If it exceeds a threshold:

$$g_i \leftarrow g_i \cdot \frac{c}{|g|}$$

This is the most common method in PyTorch training.

Engineering intuition:
• Does not truncate individual gradients
• Preserves the overall direction
• Merely reduces the step size to stabilize optimization

5. PyTorch Optimizer Design (Understanding Its Internal Structure)

An optimizer must provide:

1. Storage of parameters to be updated (param_groups)
2. A step() function to update parameters
3. A zero_grad() function to clear gradients

PyTorch only wraps these components elegantly; the essence is simple.

6. MySGD

class MySGD:
def **init**(self, params, lr=0.01, weight_decay=0.0, max_grad_norm=None):
self.params = list(params)
self.lr = lr
self.weight_decay = weight_decay
self.max_grad_norm = max_grad_norm

```
def zero_grad(self):
    for p in self.params:
        if p.grad is not None:
            p.grad.zero_()

def _clip_grad_norm(self):
    if self.max_grad_norm is None:
        return
    total_norm = torch.sqrt(sum((p.grad.norm() ** 2 for p in self.params))
    if total_norm > self.max_grad_norm:
        scale = self.max_grad_norm / (total_norm + 1e-6)
        for p in self.params:
            p.grad.mul_(scale)

def step(self):
    # 1. Clip gradients
    self._clip_grad_norm()

    # 2. Update parameters
    for p in self.params:
        if p.grad is None:
            continue

        grad = p.grad

        # Weight decay (coupled)
        if self.weight_decay != 0:
            grad = grad + self.weight_decay * p.data

        # SGD update
        p.data = p.data - self.lr * grad
```

7. Learning Rate Scheduling (Why LR Must Change Over Time)

Training has two main phases:

1. Early stage: Exploration

Requires large learning rates:

• Large update steps

• Able to escape bad regions
• Rapid decrease in loss

2. Late stage: Convergence

Requires small learning rates:
• Fine-grained adjustments
• Avoid oscillations

Thus, learning rate cannot stay constant and must be adjusted dynamically.

8. Common LR Scheduling Strategies: Formulas, Pros & Cons

(1) Step Decay

Formula:

$$\mathrm{lr}_t = \mathrm{lr}_0 \cdot \gamma^{\lfloor t/s \rfloor}$$

Pros: simple
Cons: abrupt LR changes may cause oscillations

(2) Exponential Decay

$$\mathrm{lr}_t = \mathrm{lr}_0 \cdot \gamma^{t}$$

Pros: smooth decay
Cons: may decay too quickly in late training

(3) Cosine Annealing

$$lr_t = \frac{1}{2} lr_0 \left(1 + \cos \frac{t\pi}{T}\right)$$

Pros:
• Smooth decay
• Often outperforms StepLR in practice

Cons:
• Mid-training decay may be too rapid

(4) Warmup

$$lr_t = lr_{\max} \cdot \frac{t}{T_{\text{warm}}}$$

Pros:

• Prevents early gradient explosion

9. Implementing MyStepLR

class MyStepLR:

def **init**(self, optimizer, step_size, gamma=0.1):

self.optimizer = optimizer

self.step_size = step_size

self.gamma = gamma

self.last_epoch = 0

```
def step(self):
    self.last_epoch += 1
    if self.last_epoch % self.step_size == 0:
        self.optimizer.lr *= self.gamma
```

Very intuitive: decay the learning rate once every step_size epochs.